



Smart IoT Parking System Research Report



Course: Basic Electronic

Lecturer: Ho Mony, PhD

Team Member: Moeng Kimheang

Chea Saroeurn

Pheng Virak

Phon Dyna

(2025-2026)

Contents

1. Introduction	1
1.1 Background	1
1.2 Research Problem.....	1
1.3 Research Questions	2
1.4 Proposed Framework	2
1.5 Novelty of the Study	3
1.6 Contribution of Literature Review.....	4
1.7 Aim of the Study	4
1.8 Objectives of the Study	4
2. Related Work	5
2.1 First Theory — Internet of Things (IoT) Layered Architecture	5
2.2 Second Theory — Publish/Subscribe and Command Polling for Constrained Devices.....	5
2.3 Third Theory — Optical Character Recognition (OCR) for License Plate Recognition (LPR).....	6
2.4 First Existing Research — Smart Parking with IoT Sensors and Cloud Backend	6
2.5 Second Existing Research — QR Code Payments in Transport and Parking	6
2.6 Third Existing Research — Wokwi and Digital Twin for Embedded Education	7
2.7 Research Gap	7
3. Results and Discussion.....	8
3.1 Implementation Results	8
3.2 Functional Test Results (Integration)	8
3.3 Fee Calculation (Sample)	9
3.4 Wokwi Demonstration Results.....	10
3.5 Discussion	10
3.6 Conclusion	11
References	12

1. Introduction

1.1 Background

Urban parking facilities in Cambodia and similar markets still rely heavily on manual ticketing, cash collection, and staff memory to match vehicles with payments. Drivers queue at entry and exit lanes; errors in plate recording or lost tickets cause disputes and revenue loss. At the same time, Internet of Things (IoT) devices such as microcontrollers, servos, sensors, and displays, together with mobile banking such as ABA KHQR, have become affordable for small and medium parking operators.

This project implements a Smart IoT Parking System that connects the following layers:

- Edge layer: ESP32-based gates simulated in Wokwi with servos, LCD, LEDs, and lane buttons.
- Server layer: FastAPI backend with SQLite, session and invoice logic, and a command queue for gates.
- Presentation layer: Nuxt staff dashboard and payment page with live KHQR display.

Entry uses license plate recognition with OpenCV and optional Tesseract on a lane PC. Exit uses barcode verification on the printed ticket in the format `plate|invoice|verifyHash` plus ABA payment before the exit gate opens.

1.2 Research Problem

Traditional parking operations suffer from the following problems:

1. Slow entry and exit because staff must manually write plate numbers and hand over tickets.

2. Weak traceability because there is no single database linking plate, time-in, invoice, and payment.
3. Payment friction because many exit lanes are cash-only and not integrated with national QR payment rails.
4. Disconnected IoT because gates and software are often designed separately, causing business rules to live on devices instead of a central API.

Research problem statement: *How can a low-cost IoT parking architecture combine automated plate capture at entry, cryptographically verifiable exit tickets, ABA KHQR payment, and ESP32 gate control while keeping staff dashboards read-only for safety and auditability?*

1.3 Research Questions

ID	Research question
RQ1	Can a three-tier architecture (ESP32 edge, FastAPI server, Nuxt dashboard) complete entry-to-exit parking without manual gate control from the web UI?
RQ2	Is barcode payload IOT-PARKING:plate invoice hash sufficient to verify exit identity and link to the correct invoice?
RQ3	Can ABA KHQR (mock or PayWay sandbox) update invoice status so the exit gate opens only after payment?
RQ4	Can Wokwi simulation reproduce the same API flows as real lane hardware for teaching and demonstration?

1.4 Proposed Framework

The proposed framework has four layers:

Staff dashboard (Nuxt) — read-only for gates; KHQR display
Parking API (FastAPI) — sessions, fees, invoices, payments
Gate services — entry/exit trigger, OCR, receipt print
IoT command queue — ENTRY / EXIT / DENIED
Lane PC — webcam OCR, barcode scan, thermal receipt
ESP32 (Wokwi/real) — servos, LCD, buttons, poll commands

Entry path: Lane button → POST /api/gate/entry/trigger → session + invoice + verifyHash → receipt → ENTRY_APPROVED → servo open → auto-close.

Exit path: Scan barcode → POST /api/gate/exit/trigger → fee calculation → payment (ABA/mock) → EXIT_APPROVED or EXIT_DENIED.

1.5 Novelty of the Study

For an undergraduate Basic Electronic project, the novelty is practical rather than theoretical:

1. Unified barcode contract: Code128 ticket encodes plate, invoice ID, and HMAC-derived hash in one scan at exit, so default exit flow does not require a second plate OCR.
2. Separation of concerns: the dashboard never opens gates; only IoT endpoints and lane triggers do, reducing misuse risk.
3. Simulator parity: one MicroPython firmware and one API surface work for both Wokwi buttons and real ESP32 polling.

4. Cambodia payment context: ABA KHQR on the payment page with webhook confirmation matches local mobile banking habits.
5. Teachable full stack: a single laptop can demo SQLite, mock QR, Wokwi gateway, and integration tests.

1.6 Contribution of Literature Review

This study contributes to practice by:

- Mapping IoT smart parking literature on sensors, LPR, and cloud dashboards onto an implementable student stack.
- Applying REST plus device token patterns from industrial IoT to gate command polling.
- Connecting electronic payment and QR research to parking exit gating logic.
- Documenting reproducible test and deployment steps for instructors.

1.7 Aim of the Study

To design, implement, and evaluate an integrated IoT parking system that automates entry identification, exit verification, digital payment, and gate control, with a staff dashboard for monitoring and reporting.

1.8 Objectives of the Study

#	Objective	Deliverable in repo
O1	Implement entry plate capture and session creation	gate_trigger_service.py, gate_camera_service.py, iot_entry_service.py
O2	Generate verifiable exit tickets and barcode	exit_verify_utils.py, printer_service.py / receipts
O3	Integrate exit payment and gate	payment_service.py,

	approval	iot_exit_service.py, ABA mock/PayWay
O4	Control ESP32 gates via command queue	gate_command_service.py, wokwi/parking-gate/main.py
O5	Provide staff dashboard and payment UI	frontend/ (parking, payment, invoices)
O6	Validate end-to-end flows with automated tests	scripts/test_integration.py, scripts/test_wokwi_flow.ps1

2. Related Work

2.1 First Theory — Internet of Things (IoT) Layered Architecture

Theory: IoT systems are commonly described as a stack of perception, network, processing, and application layers (Gubbi et al., 2013; Azure IoT reference models).

Relevance: Entry and exit servos and buttons are perception and actuation. Wokwi gateway and Wi-Fi connect the ESP32 to the host API. FastAPI is processing, and Nuxt is the application layer. This project deliberately keeps gate decisions in the processing layer, not on the dashboard.

2.2 Second Theory — Publish/Subscribe and Command Polling for Constrained Devices

Theory: Microcontrollers with intermittent connectivity often use message queues or HTTP long-polling rather than persistent WebSockets. MQTT is widespread in IoT literature, while HTTP polling is simpler for classroom labs.

Relevance: ESP32 firmware polls GET /api/iot/commands/next?deviceCode= and acknowledges with POST /api/iot/commands/ack. Commands are stored

in SQLite in the device_commands table until delivered, matching industrial command inbox patterns at teaching scale.

2.3 Third Theory — Optical Character Recognition (OCR) for License Plate Recognition (LPR)

Theory: LPR pipelines use image acquisition, plate localization, character segmentation, and OCR (Anagnostopoulos et al., 2008). Open-source stacks use OpenCV for capture and Tesseract or deep models for text.

Relevance: The entry lane uses GateCameraService with OpenCV webcam and optional Tesseract. Wokwi substitutes a mock plate through simulator buttons when no camera exists, while keeping the same API contract with source values such as simulator or camera.

2.4 First Existing Research — Smart Parking with IoT Sensors and Cloud Backend

Prior work: Many smart-parking papers propose occupancy sensors, mobile apps, and cloud billing, such as sensor networks in parking lots and guidance to free bays.

Comparison: This project focuses on gated lane control through servos and invoice-linked payment, not only bay occupancy. It is closer to barrier parking than street-side slot detection.

2.5 Second Existing Research — QR Code Payments in Transport and Parking

Prior work: QR-based mobile payments such as EMVCo QR and domestic schemes such as KHQR reduce cash handling. Parking systems in Asia increasingly expose QR on exit kiosks or apps.

Comparison: The payment page calls GET /api/payment/aba-qr and listens for webhook confirmation. Mock mode supports demonstrations without merchant keys, while PayWay sandbox can be configured in .env.

2.6 Third Existing Research — Wokwi and Digital Twin for Embedded Education

Prior work: Simulation environments reduce hardware cost for ESP32 labs because not every student needs physical parts.

Comparison: wokwi/parking-gate mirrors production firmware structure using MicroPython, I2C LCD, and servos. wokwigw bridges the simulator to host.wokwi.internal:8000 for real API calls during demonstrations.

2.7 Research Gap

Gap in prior practice	How this project addresses it
Gate UI mixed with payment UI	Dashboard is monitoring-only; gates are driven by the IoT API.
Exit relies only on plate re-scan	Barcode binds plate, invoice, and HMAC hash.
IoT and web built as separate assignments	Single repository with one integration test suite.
No local payment rail in student projects	ABA KHQR, webhook, and sandbox mock are included.
Hard to demo without hardware	Wokwi plus test_integration.py can run on one laptop.

Remaining gaps for future work: production PostgreSQL, JWT for staff, real ESC/POS printer, cloud LPR accuracy studies, and multi-lane scale-out.

3. Results and Discussion

Note: Full test tables and reproduction commands are in backend/README.md.

3.1 Implementation Results

The system was implemented as specified in Section 1.4. Major components are shown below.

Component	Status	Evidence
Entry trigger + session/invoice	Implemented	POST /api/gate/entry/trigger
Receipt with Code128 barcode	Implemented	backend/data/receipts/, exit_verify_utils.py
Exit trigger + barcode parse	Implemented	POST /api/gate/exit/trigger
Fee rules (<1h → \$1 min; else hourly × \$2)	Implemented	ParkingFeeService, .env rates
ABA QR + webhook	Implemented (mock default)	payment_service.py, aba_pay_service.py
ESP32 command queue	Implemented	iot router + Wokwi main.py
Dashboard + payment UI	Implemented	Nuxt parking.vue, payment.vue

3.2 Functional Test Results (Integration)

Automated test: `python -m scripts.test_integration`, with API running on `http://127.0.0.1:8000`.

Test area	Checks	Expected outcome
API health	1	status: ok

Dashboard analytics	5	JSON charts/stats
Parking & invoices lists	2	Paginated data
Bank info	1	Bank name present
Gate entry trigger	2	success, receipt path
Legacy IoT entry	1	Compatible or 409 on duplicate
Verify hash on ticket	1	Non-empty hash
Payment active session + QR	2	Plate match, data:image QR
Parking DB entry	1	Row searchable by plate
Gate exit + legacy exit	2	Fee calculated, success
Webhook + session status	2	Paid, canOpenGate
Open exit gate + invoice Paid	2	Command success, DB status
Device poll + CORS	2	200/204, localhost allowed

Target: 24/24 checks pass when the backend is running with the default .env from .env.example.

3.3 Fee Calculation (Sample)

Duration parked	Rule	Example fee (USD)
< 1 hour	Minimum fee	\$1.00
1 hour	$\text{ceil}(1) \times \$2/\text{h}$	\$2.00
2 h 10 min	$\text{ceil}(2.17) \times \$2/\text{h}$	\$6.00

Configured via MINIMUM_FEE=1.00 and RATE_PER_HOUR=2.00.

3.4 Wokwi Demonstration Results

Step	Observation
Upload firmware	Serial: boot.py: MicroPython starting → Booting... → LCD test
API online	LCD shows connectivity when backend is reachable
Green button (entry)	Entry servo opens; receipt saved; session Active
Blue button (exit)	Barcode flow; mock payment if enabled; exit servo opens
Denied exit	Unpaid or bad barcode → EXIT_DENIED, red LED + buzzer simulation

3.5 Discussion

- RQ1: Integration tests show that the API coordinates database, payment, and IoT commands without dashboard gate control, supporting the architectural objective.
- RQ2: Encoding IOT-PARKING:plate|invoice|hash allows exit to resolve the invoice in one scan. HMAC hash reduces forgery compared with plate-only tickets.
- RQ3: Mock QR and webhook paths demonstrate exit gating after Paid status. Production PayWay keys remain optional.
- RQ4: The simulator uses the same REST triggers as lane scripts, so students can demo without a physical ESP32 and then deploy the same main.py to hardware.

Limitations: OCR accuracy depends on camera and lighting. SQLite is single-user. Device tokens in .env are not rotated. Exit plate OCR is optional and the system is barcode-first by design.

3.6 Conclusion

The Smart IoT Parking System meets its stated objectives: automated entry, verifiable tickets, digital payment integration, IoT gate commands, and staff visibility. Results are reproducible through documented scripts and suitable for classroom demonstration and instructor review.

• References

- Anagnostopoulos, C.-N. E. (2014). License plate recognition: A brief tutorial. *IEEE Intelligent Transportation Systems Magazine*, 6(1), 59–67.
- Gubbi, J., Buyya, R., Marusic, S., & Palaniswami, M. (2013). Internet of Things (IoT): A vision, architectural elements, and future directions. *Future Generation Computer Systems*, 29(7), 1645–1660. <https://doi.org/10.1016/j.future.2013.01.010>
- EMVCo. (2023). *EMV QR code specifications for payment systems*. EMVCo.
- Wokwi. (n.d.). *ESP32 simulation*. Wokwi Docs. Retrieved May 30, 2026.
- Wokwi. (n.d.). *ESP32 WiFi networking*. Wokwi Docs. Retrieved May 30, 2026.
- ABA PayWay. (n.d.). *Developer suite*. ABA PayWay Developer Documentation. Retrieved May 30, 2026.
- ABA PayWay. (n.d.). *Ecommerce checkout*. ABA PayWay Developer Documentation. Retrieved May 30, 2026.